

9th Place Solution: Various Augmentations + A lot of Modeling tricks

Rank	9
Team	No Overfitting, Just Skills
Author	Mohamed Eltayeb
Topic ID	549624
Version	Original

Source: <https://www.kaggle.com/competitions/um-game-playing-strength-of-mcts-variants/discussion/549624>
Retrieved with Kaggle CLI on 2026-07-03.

Thanks to the host and Kaggle for such an amazing experience. I really enjoyed competing in this one especially with my great teammates [@cody11null](#) [@leehann](#) [@gauravbrills](#).
And finally after 3 years of competing got a gold medal and proceeded to the Master tier!!!!

Summary:

Our solution consists of ensemble between modified AdvantageP1 feature, CatBoost and two LGBM Dart models trained on different data/features.

We managed to chose our 2nd best sub in private which was 2nd in public and one of the highest in cv.

All Successful Selected Errors				Private Score ▾
Submission and Description		Private Score ⓘ	Public Score ⓘ	Selected
✓	INFER mcts_baseline_dart_tab da0994 - Version 55 Succeeded · Mohamed Eltayeb · 12d ago	0.424	0.417	☐
✓	INFER mcts_baseline_dart_tab da0994 - Version 106 Succeeded · Mohamed Eltayeb · 2d ago · best + drop aug model = cv 0.4048	0.424	0.415	☒
✓	INFER mcts_baseline_dart_tab da0994 - Version 40 Succeeded · Mohamed Eltayeb · 17d ago · best + fix all bugs = cv 0.406	0.424	0.416	☐
✓	INFER mcts_baseline_dart_tab da0994 - Version 95 Succeeded · Mohamed Eltayeb · 3d ago · best + advp1 in ensemble = cv 0.4048	0.424	0.415	☐
✓	INFER mcts_baseline_dart_tab da0994 - Version 48 Succeeded · Mohamed Eltayeb · 15d ago · best + better weights	0.424	0.416	☐
✓	INFER mcts_baseline_dart_tab da0994 - Version 107 Succeeded · Mohamed Eltayeb · 1d ago · cv 0.4029 = darts + advp1 + multirmse	0.424	0.416	☒
✓	INFER mcts_baseline_dart_tab da0994 - Version 80 Succeeded · Mohamed Eltayeb · 6d ago · best + optimized clip values	0.424	0.416	☐

We validated our models using 8-folds GroupKFold. Surprisingly, using 8 folds instead of 5 or 10 made the cv much more stable and correlated more than any other strategy.

Data Preprocessing:

1. Data Augmentation:

We explored several types of relations including the following:

Flip: Agent1=a, Agent2=b ==> Agent1=b, Agent2=a

We flip the agents, change AdvantageP1 to 1-AdvantageP1 and multiply the utility_agent1 by -1.

This is the most useful one. It worked very well for training and as TTA for inference. Especially if you trained a model for original only and a model for augmented samples only then ensemble. This gives huge boost in both cv (+~0.01 cv) and lb (+~0.003 lb) rather than merging them in the same model (+~0.001 lb).

But at the end, after many experiments we found training on it make the models less stable so dropped it from training and only used original data. Turned out to be right as that our final sub without training on Aug

performed better in private.

Self-play: Agent1=a, Agent2=b ==> Agent1=a, Agent2=a AND Agent1=b, Agent2=b

We create new samples by making the agent against himself, put Advp1 to 0.5 and utility_agent1 to 0. This helped a lot in cv but not in LB.

Transitivity: Agent1=a, Agent2=b AND Agent1=b, Agent2=c ==> Agent1=a, Agent2=c

It was very hard to setup the AdvantageP1 and utility_agent1 values for this one. I adopted this approach:

- Initialize the wins/draws/losses of the new sample to be:

```
new_row[] = row1[]

new_row[] = row2[]

new_row[] = (row1[] + row2[])
```

- Define "Skill Difference", which refer to the difference in skill between agent 1 and 3, to be:

```
diff = row1[] - row2[]
```

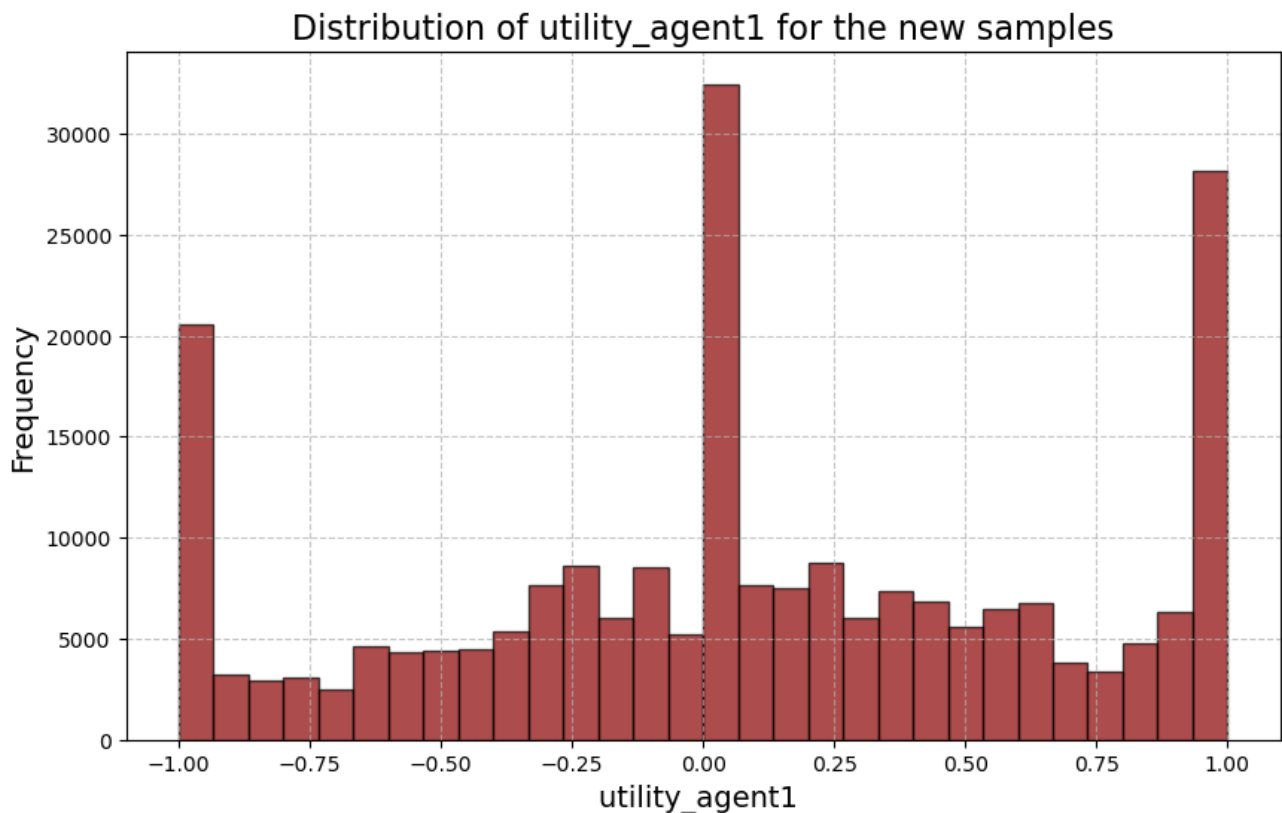
because if the middle agent in our relation (which connects the two samples) has same wins against the first and 3rd agent, then we can assume agent1 and 3 have no difference in skill, and thus we take the first wins and the the 3rd wins for our new wins and losses. But what if there is a difference? We apply this:

```
diff = row1[] - row2[]
diff < :
    new_row[] += (diff)
diff > :
    new_row[] += (diff)
```

We apply the same skill difference idea to the AdvantageP1:

```
new_row[] = row1[]
new_row[] = row2[]
diff_adv = row1[] - row2[]
clip_val = - (new_row[] + new_row[])
diff_adv < :
    new_row[] += np.clip((diff_adv),,clip_val)
diff_adv > :
    new_row[] += np.clip((diff_adv),,clip_val)
```

This actually added whole 200k new samples. Also, with the above modifications the target got distribution very similar to the original training data:



But at the end, there was no significant difference in cv from this transitivity idea, and I didn't try it against LB (I should've done after this overfitting party☺). I think it needs some more work.

Ultimately, we decided to drop all the augmentations ideas from training as they made results much less stable (we used only original data for training), and only use TTA during inference with the flip augmentation. For example, we combined the training data and augmented data in X_Train and validated against original data X_Test (to ensure correct comparison) and cv jumped from 0.405 to 0.395. 100% No leakage. But lb went from 0.416 to 0.421. At that point we believed it's easy to overfit on cv, so we tried to use ideas that improve both cv and lb only and that turned out to be the best idea at the end.

Anyway these ideas still might be valid to generate new samples from existing ones but need a bit more improvements I guess.

2. Modeling:

I and [@cody11null](#) built CatBoost and Dart model based on the above ideas. We didn't use any feature engineering or selection as they didn't help.

[@leehann](#) and [@gauravbrills](#) built Dart model based on the same ideas but with feature engineering and selection based on correlation, variance and permutation importance for the top 300 features which worked on their dart.

The first CB and Dart was giving cv 0.412 and lb 0.419. The second Dart was giving cv 0.416 and lb 0.421. Combining both gave us cv 0.406 and lb 0.416.

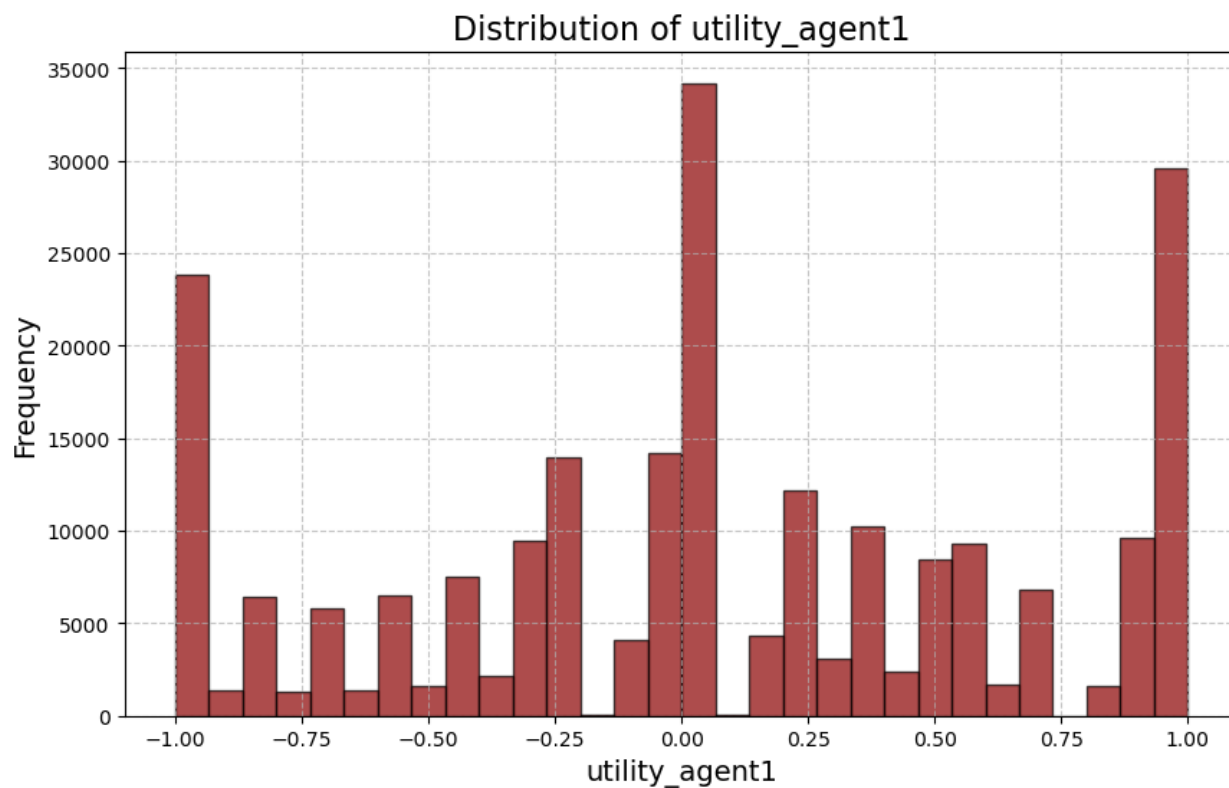
[@leehann](#) and [@gauravbrills](#) made some trials with tabnet, xgboost, clustering, embeddings but nothing

worked.

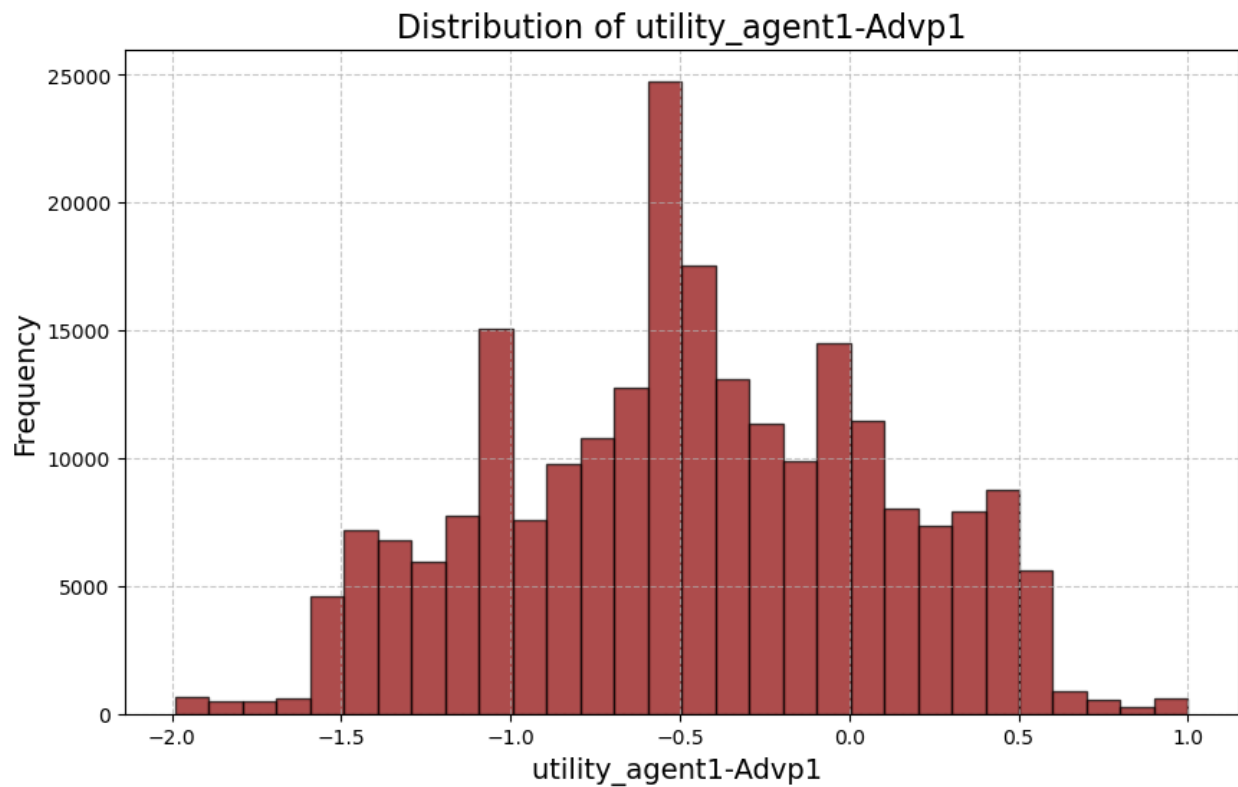
For me and [@cody11null](#), we employed several tricks:

- **Subtract AdvantageP1 from the target:**

We are predicting the following:



AdvantageP1 is very correlated to the target right? So, let's instead, predict `df["utility_agent1"] - df["AdvantageP1"]` which gives the following distribution:



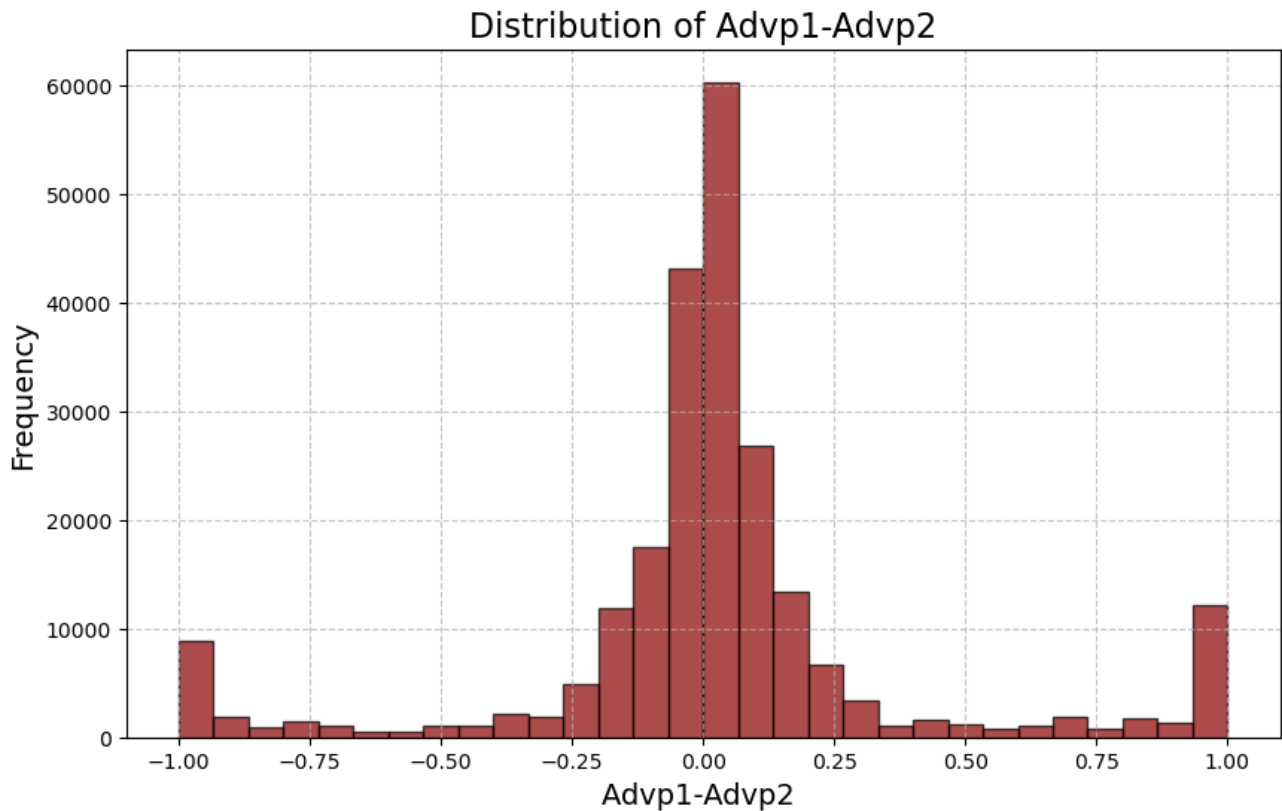
Then reverse this at prediction by adding the AdvP1 again. This actually gives me huge boost in cv ~0.02 and small one in lb <0.000xx.

- **Use AdvantageP1 in the final ensemble:**

AdvantageP1 represents the "Winning probability" right? but what we are predicting is not the winning probability. It is the winning - losing. So, easily replace AdvantageP1 as:

```
train_df[] = (train_df[] - ( - train_df[]))
```

to make it match how the target is constructed. This gives the following distribution which is similar to the target:



This didn't help as a feature in the model, but adding it to the final ensemble preds improved cv to 0.404 and lb to 0.415.

- **Predict using Wins/Losses:**

Instead of predicting the utility_agent1 instantly we tried to predict the wins/n and losses/n then subtract them. This also provided huge boost in cv ~ 0.01 but much worse lb 0.421. I thought that AdvantageP1 might be the reason for this huge boost because (similar to the last point) here we are predicting wins, and AdvantageP1 represents winning probability. So maybe because their meaning and distribution matched then the model relied too much on it? Actually this is where i get the last idea of changing the original advp1 to the difference and it worked at the end, but this wins/losses different models idea didn't work.

- **Using MultiRMSE:**

Use objective="MultiRMSE" in catboost then fit on:

```
y = pd.concat([y, -y], axis=)
```

then predict using:

```
pr = model.predict(test)
pred = (pr[:,0] - pr[:,1]) /
```

It gave a boost in cv ~ 0.002 but slightly worse result in lb 0.416. We chose this as our second sub which gave 0.424 also in private.

- **Model Per Agent:**

Tried to train a model for each agent which gave a boost in cv ~ 0.002 but same lb 0.416. got worse private 0.425.

- **Full fit:**

I found full fit on the data gave me +0.001 on lb which transitioned to private also.

- **Train a Classifier:**

Reduced the unique values of the target then used a classifier with expectation for the predictions to get to continuous values. Didn't help in cv or lb.

- **Pseudo Labeling:**

Trained my Dart on training data, then continued its training on the ensemble predictions of each 100 samples from test separately and made another new prediction. Didn't help in cv or lb.

- **Optimizing Ensemble weights:**

We optimized for $a \cdot (\text{pred}) + b$ for all the models and at the end we optimized for the clip values as well $\text{np.clip}(\text{pred}, a, b)$. Both was part of our final solution giving boosts in both cv and lb.

You can find our final inference notebook here:

<https://www.kaggle.com/code/mohammad2012191/um-inference-nb-9th-place-solution>

Supplemental Q&A

Andreas Bisiadis · 2024-12-03T14:57:01.733000

Congrats @mohammad2012191 and thanks for sharing! What does TTA stand for?

Noussair Mighri · 2024-12-03T15:04:31.217000

I think TTA is Test-Time Augmentation which consist of applying augmentations to the input data during inference and combining the predictions from the augmented inputs but I can't understand how we can use it for this case

Cody_Null · 2024-12-03T15:18:05.270000

Yes it is Test Time Augmentations and it is augmented in the same way as the second set of models. That is flipped agents.

Andreas Bisiadis · 2024-12-03T16:05:48.063000

@cody11null

1. How did augmenting the test data improve the LB? I can't figure out why making predictions on a larger set of data than the "asked" one boosts the LB.
2. Was the 10 minutes per batch limit an issue for TTA?

Mohamed Eltayeb · 2024-12-03T17:44:53.067000

1.

Original sample: agent1 = a, agent2 = b, AdvantageP1 = v ==> Output: utility_agent1

Augmented (Flipped) sample: agent1 = b, agent2 = a, AdvantageP1 = 1 - v ==> Output: utility_agent2 = - utility_agent1

```
final pred = 0.5*Output1 + 0.5*(-Output2)
```


2.

We are not training. We are just making predictions on augmented sample then multiply its output by -1 to get `utility_agent1`. Then average that with the original sample prediction.

It is like making the model make the same prediction from two different perspectives, which increased diversity obviously.